Informalizing Lean Mathlib

Mathlib in English and French with Informath, built with Claude Code

Aarne Ranta

22 October 2026

Lean and Mathlib

- **Lean 4**: a proof assistant and functional programming language based on dependent type theory (de Moura et al.; now the Lean FRO).
- **Mathlib**: Lean's community library of formalized mathematics, from algebra and number theory to analysis, topology and category theory.
- At the commit used here (`c31114ff81`, 21 Oct 2026): 8,650 files, 2.3 million lines, about 194,000 theorems and 48,000 definitions, structures and classes.
- Statements are read by Lean's *elaborator*: type classes, implicit arguments, coercions and notation are resolved there, not in the source text.

```
/-- ‘Nat.Prime p‘ means that ‘p‘ is a prime
     number,
  that is, a natural number at least 2 whose only
  divisors are ‘p‘ and ‘1‘. -/
@[pp_nodot, wikidata Q49008, oeis A000040]
def Prime (p : ℕ) :=
  Irreducible p

theorem exists_prime_and_dvd {n : ℕ} (hn : n ≠
     1) :
    ∃ p, Prime p ∧ p ∣ n :=
  ⟨minFac n, minFac_prime hn, minFac_dvd _⟩
```

Mathlib/Data/Nat/Prime/Defs.lean

The problem: Mathlib is formal

What informal text there is

- **Docstrings**, written by hand. Batteries' `docBlame` linter requires them for definitions; `docBlameThm`, the same for theorems, is disabled. In our checkout, **83%** of definitions, structures and classes have one, but only **15%** of theorems and lemmas.
- **Module docstrings** summarizing each file, and the HTML documentation generated by doc-gen4, which shows the formal statements with the docstrings.
- **LLM informalizations** made for search and for training: LeanSearch (Gao et al., EMNLP Findings 2024) and the Herald dataset (Gao et al., ICLR 2025), checked by back-translation.

What is missing

- Readings of *all* statements that are faithful by construction, reproducible, and in several languages.
- Proofs in natural language: none of the above covers them.

GF and Informath

GF, Grammatical Framework

- An abstract syntax (trees) with concrete syntaxes (linearizations) for many languages.
- The Resource Grammar Library takes care of morphology and syntax: agreement, word order, inflection.

Informath (2024–, 932 commits)

- Translates between **Dedukti** and natural language, in both directions, with LaTeX formulas.
- A Dedukti dialect: `Set`, `Elem`, `Prop`, `Proof`, and a base of arithmetic and logic.
- **Symbol tables** (`.dkgf`) give each constant its verbal and symbolic forms.

From Dedukti to text

1. Annotate the constants with the symbol tables.
2. Translate to *MathCore*, a minimal GF syntax.
3. Generate the variants: verbal or symbolic, aggregated, in-situ quantifiers...
4. Linearize them in English, French, Swedish, ... or as LaTeX formulas.
5. Rank them, and pick the best.

Proof texts (`-proof-text`): numbered lines, each justified by a rule or a lemma, as in godement-index.

Building mathlib-informath

Two days, nine prompts

	prompt (abridged)	outcome	commits
1	build an informalization system for Mathlib	exporter, Nat primes	`c46c6c3`
2	build mathlib.dkgf and apply it	17 entries: 69 → 7 unworded	
3	an HTML page with Lean, Dedukti, MathJax, GitHub links	the page, then GitHub Pages	
4	split it into subfiles; Factorization next	322 statements on 2 pages	`3e8bc29`
5	keep the definientia, show the proofs	84 proof texts; Informath fix	`5e78612` `1658da6`
6	natural language from the sources	docstrings and names; `Mathlib.gf`	`8244cae`
7	macros in red: MathJax issue?	a builder bug; $\mathrm{ord}_p(n)$	`6b7383c`
8	the Algebra and Order.Defs modules	705 statements on 4 pages	`976bbd0`
9	add French	a third row, an index	`823155a`

21–22 October 2026: 8 commits in mathlib-informath and 4 in informath (`4f31f55`, `ceb9b85`, `7773194`, `778e998`), all made by Claude when asked, and pushed by me.

1. The plan: three decisions

> *"you can see ../mathlib4, right? It is Leans famous Mathlib. I would like to build an informalization system for it using Informath."*

Claude surveyed Mathlib, Informath's Lean support and lean2dk (Deducteam's faithful Lean-to-Dedukti translator), then asked three questions:

1. **Input:** own Lean exporter (recommended), lean2dk, or parsing the Lean source. lean2dk encodes the whole type theory, which is the opposite of what Informath reads; the source text lacks what the elaborator adds.
2. **First slice:** Algebra/Group/Basic, Nat primes, or orders and lattices.
3. **Setup:** a new project, with Mathlib's build cache downloaded (7 GB in two minutes, instead of hours of compiling).

The answers are in green. The first run read all 130 statements of the five prime modules back into English, 61 of them already fully worded.

The exporter: from Lean's environment to Dedukti

`Export.lean` (now 500 lines) runs over Mathlib's elaborated environment:

- binders become `Elem`, `Proof` or `Set` hypotheses;
- implicit and instance arguments are dropped from applications;
- casts and coercions are the identity, numerals Informath's digits;
- `mapping.tsv` maps Lean's notions to Informath's base: `Nat.Prime`→`prime`, `Dvd.dvd`→`divide`;
- per module: `.dk`, `.json` (docstrings, lines), `-deps.dk`.

```
theorem Prime.eq_one_or_self_of_dvd {p : ℕ} (pp : p.
     Prime)
    (m : ℕ) (hm : m ∣ p) : m = 1 ∨ m = p
```

```
Nat__Prime__eq_one_or_self_of_dvd : (p : Elem Nat) ->
  (pp : Proof (prime p)) -> (m : Elem Nat) ->
  (hm : Proof (divide m p)) -> Proof (or (Eq m 1) (Eq
      m p)).
```

Let p be a natural number. Assume that p is prime. Let m be a natural number. Assume that m divides p. Then $m = 1$ or $m = p$.

$p \in \mathbb{N}, \mathrm{prime}(p), m \in \mathbb{N}, m \mid p \vdash m = 1 \vee m = p$

2. A symbol table for Mathlib's constants

"build mathlib.dkgf and apply it"

- 17 entries, worded after Mathlib's docstrings where the grammar had the words, checked against the exported types: `Nat.minFac` "the minimal prime factor of #1", `Nat.Coprime` "#1 and #2 are coprime" | $\#1 \perp \#2$.
- Readings with an unworded constant: **69** $\rightarrow$ **7** of 130.

Writing the table exposed three exporter bugs

- Prop-valued constants were typed `Set`.
- Lean's `Set` clashed with Informath's: it now maps to `powerset`.
- Definitions were declared with implicit arguments that the applications drop, so `#k` counted the wrong arguments.

And Informath issues, reported but not fixed

$ inside a quoted verbal form splits a table entry; a proposition inside a macro argument loses its notation; "does not have a upper bound".

3. The page, and GitHub Pages

> *"produce a mathlib-informath.html file, which shows for each statement the original lean, the dedukti translation, and the informalization in mathjax latex, as well as a link to the original lean source in mathlib github"*

- Each entry: name, kind, docstring, the Lean source up to its `:=`, the Dedukti, English, the formula, and a link to the line at Mathlib's exact commit.
- The exporter now records source positions and keeps the source order. Lean-generated declarations, which have no position, are left out.
- Checked in headless Chrome: MathJax renders the readings.

> *"will it be hosted in github?"*

Not yet: the project had no remote. Claude explained the two steps, a repository and GitHub Pages, but left both to me, as they publish under my name. I created `aarneranta/mathlib-informath`; Claude committed `c46c6c3`; I pushed.

4. More of Mathlib: Factorization, and a page per part

"Do more of Mathlib. But now you should split the html document to a main file, which contains a table of contents with links to subfiles of reasonable size. ... Factorization is perhaps the next example?"

- One page per directory; the table of contents forwards old links (`#Nat.Prime.two_le`) to the right page, since the page is public.
- **Coercions to functions:** `n.factorization p` becomes one constant `Nat__factorization__app n p`.
- **Binders** ($\prod_{x \in s} f(x)$, $\{x \in s \mid P\}$): the exporter eta-expands the function arguments, and raw macros take the bound variable as a parameter, as Informath's `\Summa`.
- **Type classes** become predicates: `[CommMonoid M]` reads "Assume that M is a commutative monoid".
- About 30 new table entries; deprecated aliases left out; a `-force-symbolic` fallback for statements with no reading.

Result: 322 statements on two pages, 267 fully worded (`3e8bc29`).

5. Definientia and proofs

"I don't think we need to cut judgements at := if the definiens is an explicit expression directly translatable to Dedukti. Keep it in those cases, and if the value type is Proof X, show the proof in the same way as in godement-index."

- Only term definientia (not by tactics, not equations). Theorems' proofs needed `value`? (`allowOpaque := true`).
- Proofs go into the natural deduction rules of `nd.dk`: λs proving $\to$ or $\forall$ become `ifI`/`forallI`, applied hypotheses `ifE`/`forallE`.
- Lemmas keep their arguments in proofs, except type class instances.
- An Informath fix (`4f31f55`): `-proof-text` crashed on a λ typed by a proof of an implication.

```
theorem exists_prime_and_dvd {n : ℕ} (hn : n ≠
    1) :
  ∃ p, Prime p ∧ p ∣ n :=
⟨minFac n, minFac_prime hn, minFac_dvd _⟩
```

1.	Let n be an element of $\mathbb{N}$.	
2.	Assume that $n \neq 1$.	
3.	minFac(n) is prime.	`minFac_prime 2`
4.	minFac(n) $\mid n$.	`minFac_dvd`
5.	minFac(n) is prime and minFac(n) $\mid n$.	`andI 3, 4`
6.	There exists an element p of $\mathbb{N}$, such that p is prime and $p \mid n$.	`existsI 5`

6. Natural language from the sources

> *"find more natural language expressions instead of verbatim Lean constants. . . . Consider for instance primeFactorsList(n) . . . Could this information be found somewhere in the available sources?"*

Three sources, as Claude found them:

- **docstrings**, in fixed patterns: "`f n` is the X of `n`", "Returns the X. . . ";
- **names**, by Mathlib's naming conventions: `IsUnit` → "#1 is a unit";
- **Wikidata attributes**, but only on a few declarations.

`suggest-entries.py` makes candidates from the first two and keeps those that `-parse-example` parses.

> *"if you need new words, add them to Mathlib.gf"*

- 31 reviewed entries: "the smallest prime factor of #1", "the set of prime factors of #1".
- `Mathlib.gf`: "smallest", "descending", "bounded above", "set of divisors". . . (`ceb9b85`).
- English is read from a table without pseudo-notations such as primeFactors(n), which the ranking preferred.

Factorization readings with a raw Lean name: **94** → **8** of 194.

7. Red macros

"In Factorization.html, some macros are rendered in red and not expanded. For instance, \NatfactorizationappMACRoI. Is this a MathJax Issue or something easy to fix?"

- Not MathJax: Claude's own bug from step 6. Macros are numbered by their position in the table, and the English table, with fewer LaTeX forms, numbered them otherwise. 37 macros were undefined.
- The fix: the pages define the macros of both tables, with a check for clashing names (none).
- The screenshot check found another problem: the multiplicity of p in n and the p-adic valuation were written $\nu_p(n)$ and $v_p(n)$. Claude asked; I chose to commit both changes, and the multiplicity is now $\mathrm{ord}_p(n)$ (6b7383c).

8. Down the imports: Algebra and Order

> *"I can see that already Prime.Defs imports some Algebra modules: Group, Prime, etc. Can you run informalization with those Algebra modules and also Order.Defs modules?"*

- The five `Order.Defs` modules (354 statements) and two Algebra modules; `GroupWithZero.Nat` has only instances and was left out.
- `# page: X` lines in `modules.txt` put small modules on one page.
- About 40 entries for order, algebra and relation classes: "#1 is partially ordered", "#2 is a strict weak order", "the supremum of #1 and #2" | #1 ⊔ #2; four more words in `Mathlib.gf`.
- `Decidable` instance hypotheses are computational, so the exporter omits them.
- "*r* holds for *a* and *b*", a relation *variable*, now counts as worded.
- The slow proof texts (`sup_assoc`) got a time limit.

Result: 705 statements on four pages, 574 fully worded (`976bbd0`).

9. French

"Now, let us add French! Can you figure out what is needed?"

Claude's findings:

1. the full grammar was six weeks old;
2. French proof texts were *empty*: `ProofUnitsFre` lacked the two functions that proof lines are made of;
3. 27 words lacked French, in 5 modules;
4. the page builder needed a French row;
5. grammar flaws, for later.

"Do 2-4 and add French as a third row for the time being"

`le_antisymm`:

Let x be a set. Assume that x is partially ordered. Let a and b be elements of x. Assume that $a \leq b$. Assume that $b \leq a$. Then $a = b$.

Soit x un ensemble. Supposons que x est partiellement ordonné. Soient a et b des éléments de x. Supposons que $a \leq b$. Supposons que $b \leq a$. Alors $a = b$.

9. French: two interruptions

The first build took minutes: every French call loads the full grammar (2.8 s), and the proofs were read one at a time.

> *"It is better to build an index that maps Dedukti to French and English and LaTeX in one go, so that the grammar is only loaded once. Then use this index to populate the html."*

- Informath (`778e998`): `-languages=Eng,Fre` for `-json` and `-proof-text`, and `-no-back-to-dedukti`, as the back-translation crashed on proofs.
- `build-index.py` → `index.json` in **27 s**, and `build-html.py` renders it in 0.06 s.
- Keyed by Dedukti names, the index revealed that the old pages had put some readings on the wrong declarations.

> *"don't add german yet, just French and English"*

Claude had rebuilt the grammar with German, to keep a regression test running; the full grammar now has English and French only (`823155a`).

The result

page	statements	fully worded, English	French
Mathlib.Order.Defs	354	281	281
Mathlib.Algebra	29	26	26
Mathlib.Data.Nat.Prime	128	125	125
Mathlib.Data.Nat.Factorization	194	142	142
all	**705**	**574**	**574**

- 264 declarations with their definiens; 240 proof texts in each language, out of 242 term proofs.
- Every statement has a formula; 31 need `-force-symbolic`.
- `https://aarneranta.github.io/mathlib-informath/mathlib-informath.html`

Open problems

Open problems: coverage

- **Scale.** 705 of Mathlib's 240,000 declarations, with 108 hand-reviewed table entries. All of Mathlib needs entries generated in bulk from docstrings and names, verified by the grammar and the type checker.
- **Tactic proofs.** 378 of the 620 theorems here, and most of Mathlib's, are proved by tactics. They need the proof term that Lean elaborates, or a reading of the tactic script.
- **Unworded statements** (131 of 705):
  - multi-letter variables (`lt`, `self`, `motive`) are shown as lt and counted as constants;
  - function arguments: `Set.ofPred`, the motives of induction;
  - `Leq` used unapplied: $\mathrm{Leq}(x_2, x_3)$.
- Decidability instances defined as declarations are still exported; two `Decidable` proofs (with `dite`) have no proof text.
- Theorem labels are mangled names: `Nat__Prime__two_le`.

Open problems: language

- **Proof texts** where a lemma or constructor is used unapplied: "For all sets x and functions r … and elements irrefl of Proof applied to the proposition that …, r is irreflexive." Bound variables are renumbered (x_4, x_5) where the source reuses names.
- **Meaning that depends on the type**: $\leq$ on finite sets reads "less than or equal to", where Mathlib means "subset of".
- **French**: "pour tous des nombres naturels", "il n'est pas le cas que" (a calque of "it is not the case that"), "une relation d'équivalences"; three French modules are partial.
- **Informath**:
  - `$` in a quoted verbal form splits a table entry (`splitEntry`), which rules out binder shapes in tables;
  - a proposition inside a macro argument loses its notation;
  - "does not have a upper bound";
  - `proof2dedukti` does not cover proofs, so the back-translation of `-json` crashes on them.
- Swedish and German are left out of the full grammar for now.

Summary

Who did what

Already in Informath	From my prompts	Added by Claude
The GF grammar and its RGL-based concrete syntaxes, French among them (2025–; `WikidataWordsFre` completed with Claude in July)	The goal: an informalization system for Mathlib with Informath	`Export.lean`: the translation rules, classes as predicates, `__app` coercions, eta-expanded binders, proofs in `nd.dk` rules, filters
The Dedukti dialect, its base constants, symbol tables with verbal, symbolic and raw macro forms	The choices among Claude's options: own exporter, Nat primes first, a new project with the cache	`mapping.tsv`; `mathlib.dkgf` (108 entries) with `suggest-entries.py`
NLG with variants and ranking; `-to-symbolic-latex`, `-force-symbolic`, `-parse-example`	The page's contents; GitHub hosting (I created the repository and pushed)	`build-index.py`, `build-html.py`: index, pages, macros, proof links
`-proof-text` (since June, 8 of 21 commits with Claude), and godement-index's way of showing proofs	A page per part; keeping the definientia and showing the proofs; words from the sources; new words in `Mathlib.gf`	Words in `Mathlib.gf`, `MathlibFre.gf`, and partial French modules
The regression suite, the autoformalization template	The next modules; French as a third row; one index loading the grammar once; no German yet	Informath: `abstractionTypes`, `-languages`, multilingual proof texts, `-no-back-to-dedukti`, `ProofUnitsFre`

Claude also found and reported the bugs along the way, its own among them (the red macros, the misaligned readings), and asked before committing; I pushed.

`github.com/aarneranta/mathlib-informath`

705 Mathlib statements, 240 proofs, in English, French and LaTeX